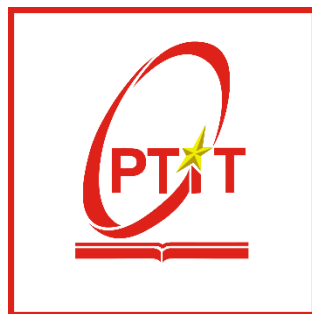


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

-----o0o-----



BÁO CÁO ĐỒ ÁN CUỐI KỲ

MÔN: CÁC HỆ THỐNG PHÂN TÁN

**CHỦ ĐỀ 2: PHÁT TRIỂN HỆ THỐNG SOẠN THẢO VĂN BẢN
CỘNG TÁC (COLLABORATIVE DOCUMENT EDITING SYSTEM)**

Giảng viên : Kim Ngọc Bách

Nhóm lớp : 03

Nhóm bài tập lớn: : 16

Thành viên nhóm :

Bùi Xuân Trường- B22DCCN878

Hoàng Đình Phong – B22DCCN614

Nguyễn Duy Hải Đăng – B22DCCN208

Hà Nội - 2026

MUC LUC

I. Giới thiệu	4
1. Mục tiêu đồ án	4
2. Yêu cầu chức năng	4
3. Yêu cầu kỹ thuật	6
4. Chức năng nâng cao	6
5. Công nghệ sử dụng	6
II. Kiến trúc hệ thống	7
Kiến trúc phân tầng (Layered Architecture)	7
Luồng xử lý thao tác	8
1. Phát sinh và Khởi tạo Thao tác tại Client (Client Upstream)	8
2. Tiếp nhận và Kiểm tra Điều kiện tại Server (Server Ingestion)	8
3. Biến đổi Thuật toán và Lưu trữ tại Server (Server Transformation & Persistence)	8
4. Phản hồi và Đồng bộ hóa Toàn cục (Downstream Broadcast)	9
III. Mô hình đồng bộ dữ liệu	9
1. Mô hình kênh sự kiện (Event-Driven Architecture)	9
2. Mô hình dữ liệu tập trung (Centralized Data Model)	10
IV. Giao thức trao đổi dữ liệu	11
1. Giao thức RESTful API	11
2. Giao thức WebSocket	12
V. Kiểm thử và đánh giá	14
Các trường hợp thử nghiệm chính	14
Đánh giá kết quả	17

I. Giới thiệu

1. Mục tiêu đề án

Hệ thống soạn thảo văn bản cộng tác (Collaborative Document Editing System) cho phép nhiều người dùng cùng chỉnh sửa một tài liệu đơn cùng lúc với việc đồng bộ thay đổi theo thời gian thực. Hệ thống kết hợp các công nghệ web hiện đại như React, Node.js, Socket.IO và MongoDB(Atlas) để tạo nên một nền tảng đáng tin cậy và hiệu quả.

2. Yêu cầu chức năng

Hệ thống hỗ trợ các chức năng cơ bản sau:

1. Tạo và quản lý tài liệu

Mô tả:

Hệ thống cung cấp giao diện cho phép người dùng tạo, xem, và quản lý các tài liệu văn bản.

Chi tiết chức năng:

- Người dùng có thể tạo tài liệu mới với tiêu đề và nội dung ban đầu
- Hệ thống hiển thị danh sách tất cả tài liệu người dùng có quyền truy cập
- Người dùng có thể mở tài liệu để xem và chỉnh sửa
- Người dùng có thể xóa tài liệu của mình (tùy chọn: yêu cầu xác nhận)
- Hiển thị thông tin tài liệu: tác giả, ngày tạo, ngày sửa đổi lần cuối

2. Chỉnh sửa cộng tác theo thời gian thực

Mô tả:

Nhiều người dùng có thể đồng thời chỉnh sửa cùng một tài liệu, với mọi thay đổi được phát tán tức thời.

Chi tiết chức năng:

- Khi người dùng nhập vào trình soạn thảo, các ký tự được gửi tới server qua Socket.IO
- Server nhận và phát tán các thay đổi tới tất cả client khác đang mở cùng tài liệu
- Độ trễ cập nhật không vượt quá 500ms (gần thời gian thực)

- Hỗ trợ các thao tác cơ bản: nhập ký tự, xóa ký tự, sao chép, dán, cắt
- Hỗ trợ định dạng cơ bản (tùy chọn): in đậm, in nghiêng, gạch chân
- Lưu tự động (autosave) các thay đổi mà không yêu cầu người dùng bấm nút lưu

3. **Đồng bộ thay đổi**

Mô tả:

Hệ thống đảm bảo tính nhất quán của tài liệu khi nhiều người chỉnh sửa đồng thời.

Chi tiết chức năng:

- Áp dụng Operational Transform (OT) để quản lý và điều hòa các phiên chỉnh sửa đồng thời từ phía người dùng
- Mỗi thay đổi được gán một số thứ tự (sequence number) duy nhất
- Khi có xung đột, server xử lý các thao tác dựa trên thuật toán OT để duy trì tính nhất quán
- Đảm bảo nội dung tài liệu giống nhau trên tất cả client bất kể thứ tự nhận thay đổi
- Cung cấp cơ chế khôi phục (undo) với hỗ trợ cho tài liệu cộng tác
- Hỗ trợ lịch sử chỉnh sửa để người dùng có thể xem các phiên bản cũ

4. **Quản lý người dùng đang chỉnh sửa**

Mô tả:

Hệ thống theo dõi và hiển thị thông tin về các người dùng đang tham gia chỉnh sửa.

Chi tiết chức năng:

- Hiển thị danh sách người dùng hoạt động đang chỉnh sửa tài liệu hiện tại
- Mỗi người dùng được gán một màu sắc duy nhất và hiệu ứng trên giao diện
- Hiển thị vị trí con trỏ của từng người dùng trong tài liệu
- Hiển thị tên người dùng khi di chuột trên con trỏ của họ (tooltip)
- Cập nhật tình trạng người dùng khi họ tham gia hoặc rời khỏi phiên chỉnh sửa
- Hỗ trợ hiển thị khu vực được chọn của từng người dùng
- Thông báo khi có người dùng mới tham gia hoặc rời khỏi tài liệu

5. **Lưu trữ tài liệu**

Mô tả:

Tài liệu được lưu trữ bền vững và an toàn trên server.

Chi tiết chức năng:

- Tất cả tài liệu được lưu trong MongoDB Atlas với cấu trúc dữ liệu rõ ràng
- Mỗi tài liệu có các trường: ID, tiêu đề, nội dung, tác giả, ngày tạo, ngày sửa đổi
- Autosave sau mỗi thay đổi hoặc theo khoảng thời gian định kỳ (ví dụ: 2 giây)
- Đảm bảo dữ liệu không bị mất nếu client ngắt kết nối đột ngột
- Hỗ trợ cơ chế sao lưu tài liệu định kỳ
- Kiểm soát quyền truy cập: chỉ tác giả hoặc người được chia sẻ mới có thể truy cập

3. Yêu cầu kỹ thuật

Hệ thống thể hiện rõ các yếu tố của hệ thống phân tán:

- giao tiếp mạng giữa client và server (WebSocket)
- xử lý **nhiều client đồng thời**
- cơ chế **đồng bộ thay đổi giữa các client**
- xử lý các xung đột chỉnh sửa đồng thời

4. Chức năng nâng cao

Hệ thống được triển khai thêm một số chức năng:

- lịch sử chỉnh sửa và **versioning**
- **undo/redo** trong môi trường cộng tác
- hiển thị **con trỏ và vùng chọn của người dùng khác**
- cơ chế **offline editing và đồng bộ lại khi kết nối lại**
- phân quyền người dùng (owner, editor, viewer)

5. Công nghệ sử dụng

Frontend	React 19, Vite, Socket.IO client
Backend	Node.js, Express, Socket.IO, Mongoose
Database	MongoDB / MongoDB Atlas

II. Kiến trúc hệ thống

Trong đồ án này, hệ thống văn bản cộng tác được tổ chức theo **kiến trúc phân cấp (kiến trúc đọc)** dưới dạng mô hình hình cây để tối ưu hóa việc quản lý luồng dữ liệu và phân quyền người dùng. Cấu trúc hệ thống được chia thành nhiều cấp độ với vai trò trung tâm dịch chuyển dần lên các nút phía trên:

- **Gốc (Database/Model):** Lưu trữ trạng thái bền vững (Persisted State).
- **Thân/Nhánh (SocketServer):** Tách trạng thái động theo từng Room ID. Server đứng ở giữa quản lý phân nhánh, đảm bảo operation của Room tài liệu A không bị gửi nhầm sang Room tài liệu B.
- **Lá (Client Sockets):** Nhận dữ liệu hạ lưu được lọc sạch từ nhánh (Room) mà nó trực thuộc.

Kiến trúc phân tầng (Layered Architecture)

Kiến trúc phân tầng (Layered Architecture) là mô hình thiết kế phần mềm chia hệ thống thành nhiều tầng chức năng độc lập. Mỗi tầng đảm nhận một vai trò riêng và chỉ giao tiếp với các tầng liên quan nhằm giảm sự phụ thuộc giữa các thành phần trong hệ thống.

Thông thường hệ thống được chia thành các tầng:

- Tầng giao diện (Presentation Layer): hiển thị dữ liệu và tương tác với người dùng.
- Tầng xử lý nghiệp vụ (Business Logic Layer): xử lý logic và quy tắc của hệ thống.
- Tầng truy cập dữ liệu (Data Access Layer): trung gian giữa ứng dụng và cơ sở dữ liệu.
- Tầng cơ sở dữ liệu (Database Layer): lưu trữ và quản lý dữ liệu.

Mô hình này giúp hệ thống dễ bảo trì, dễ mở rộng và thuận lợi cho việc phát triển theo nhóm. Ngoài ra, việc tách biệt chức năng giữa các tầng còn giúp tăng khả năng tái sử dụng mã nguồn và hỗ trợ kiểm thử độc lập từng thành phần.

Tuy nhiên, do dữ liệu phải đi qua nhiều tầng xử lý nên có thể làm tăng độ trễ và khiến luồng xử lý phức tạp hơn trong các hệ thống lớn.

Luồng xử lý thao tác

1. Phát sinh và Khởi tạo Thao tác tại Client (Client Upstream)

Khi người dùng tương tác và thay đổi nội dung trên giao diện, hệ thống phía client sẽ tự động thực hiện thuật toán so khớp (Diff) để xác định chính xác phần nội dung bị biến đổi. Thay đổi này được đóng gói thành một đối tượng thao tác (Operation), đi kèm với số hiệu phiên bản nền tảng tại thời điểm đó (Base Revision). Thao tác này sau đó được đánh dấu là "đang xử lý" (In-flight Operation) để khóa luồng gửi tạm thời và được truyền lên máy chủ thông qua kênh kết nối thời gian thực.

2. Tiếp nhận và Kiểm tra Điều kiện tại Server (Server Ingestion)

Ngay khi tiếp nhận thao tác từ client, máy chủ thiết lập một hàng rào kiểm tra toàn vẹn bao gồm:

- **Kiểm tra quyền truy cập:** Xác thực danh tính và vai trò của người dùng (Role) để đảm bảo họ có quyền ghi/sửa đổi tài liệu.
- **Xác thực cấu trúc:** Kiểm tra tính hợp lệ của gói tin operation được gửi lên.
- **Xác định xung đột cạnh tranh:** Máy chủ tải lịch sử tài liệu từ cơ sở dữ liệu để so sánh. Nếu số hiệu phiên bản hệ thống hiện tại cao hơn số hiệu phiên bản nền tảng của client gửi lên, máy chủ sẽ xác định được danh sách các thao tác đồng thời (Concurrent Operations) đã được áp dụng trước đó bởi người dùng khác mà client này chưa kịp cập nhật.

3. Biến đổi Thuật toán và Lưu trữ tại Server (Server Transformation & Persistence)

Để giải quyết xung đột, máy chủ đưa thao tác của client và danh sách các thao tác đồng thời vào bộ xử lý biến đổi tuần tự (Operational Transformation). Thuật toán sẽ tính toán và điều chỉnh lại chỉ mục (index) của thao tác mới sao cho phù hợp với trạng thái hiện tại của văn bản. Sau khi có thao tác biến đổi chuẩn, máy chủ thực hiện:

- Áp dụng trực tiếp thay đổi vào văn bản gốc trên server.
- Tăng số hiệu phiên bản toàn cục của tài liệu (Global Revision).
- Ghi nhận thao tác mới vào nhật ký lịch sử thay đổi (Operation Log).
- Lưu trữ lâu dài nội dung mới và số hiệu phiên bản vào cơ sở dữ liệu để đảm bảo tính bền vững của dữ liệu.

4. Phản hồi và Đồng bộ hóa Toàn cục (Downstream Broadcast)

Cuối cùng, máy chủ phân phối kết quả xử lý xuống hạ lưu thông qua cơ chế phân tách phòng (Room) để cập nhật trạng thái cho toàn hệ thống:

- **Đối với Client gửi thao tác:** Máy chủ gửi một tín hiệu xác nhận (Acknowledgment - Ack). Client nhận được tín hiệu này sẽ giải phóng thao tác đang chờ, cập nhật nội dung chuẩn tối hậu và chuyển sang số hiệu phiên bản mới.
- **Đối với các Client khác trong cùng phòng:** Máy chủ phát sóng (Broadcast) thao tác đã qua biến đổi đến các client này. Khi tiếp nhận, phía client sẽ tự động biến đổi thao tác từ xa (Remote Operation) này với các thao tác cục bộ hiện tại của mình để cập nhật giao diện hiển thị, đồng thời tự động điều chỉnh lại các ngăn xếp hoàn tác/làm lại (Undo/Redo Stacks).

III. Mô hình đồng bộ dữ liệu

1. Mô hình kênh sự kiện (Event-Driven Architecture)

Mô hình kênh sự kiện (Event-Driven Architecture) là mô hình mà các thành phần trong hệ thống giao tiếp với nhau thông qua các sự kiện (Events). Khi một hành động xảy ra, hệ thống sẽ phát sinh một sự kiện và các thành phần liên quan sẽ lắng nghe và xử lý sự kiện đó.

Trong mô hình này tồn tại ba thành phần chính:

- Event Producer: thành phần tạo ra sự kiện.
- Event Channel/Broker: kênh truyền tải sự kiện.
- Event Consumer: thành phần nhận và xử lý sự kiện.

Mô hình event-driven đặc biệt phù hợp với các hệ thống thời gian thực, hệ thống phân tán và các ứng dụng cần xử lý bất đồng bộ. Việc sử dụng sự kiện giúp giảm sự phụ thuộc trực tiếp giữa các thành phần, tăng khả năng mở rộng và cải thiện hiệu năng xử lý đồng thời.

Ngoài ra, mô hình này còn giúp hệ thống phản hồi nhanh hơn đối với các thay đổi xảy ra liên tục trong môi trường nhiều người dùng.

Tuy nhiên, do luồng xử lý diễn ra bất đồng bộ nên việc theo dõi và gỡ lỗi hệ thống trở nên khó khăn hơn. Khi số lượng sự kiện lớn, việc quản lý thứ tự xử lý và đồng bộ dữ liệu cũng phức tạp hơn.

2. Mô hình dữ liệu tập trung (Centralized Data Model)

Mô hình dữ liệu tập trung là mô hình trong đó toàn bộ dữ liệu của hệ thống được lưu trữ và quản lý tại một máy chủ hoặc trung tâm dữ liệu duy nhất. Các client sẽ truy cập và thao tác dữ liệu thông qua máy chủ trung tâm này.

Mô hình dữ liệu tập trung giúp đảm bảo tính nhất quán dữ liệu vì mọi thay đổi đều được xử lý tại cùng một nơi. Việc quản lý, sao lưu và phục hồi dữ liệu cũng trở nên dễ dàng hơn do dữ liệu không bị phân tán trên nhiều nút khác nhau.

Trong các hệ thống cộng tác hoặc hệ thống quản lý tập trung, mô hình này giúp kiểm soát quyền truy cập, đồng bộ trạng thái và duy trì toàn vẹn dữ liệu hiệu quả hơn.

Ngoài ra, việc triển khai và bảo trì hệ thống cũng đơn giản hơn so với các mô hình dữ liệu phân tán.

Tuy nhiên, mô hình dữ liệu tập trung tồn tại nhược điểm là máy chủ trung tâm có thể trở thành điểm nghẽn của hệ thống khi số lượng truy cập tăng cao. Nếu máy chủ trung tâm gặp sự cố, toàn bộ hệ thống có thể bị ảnh hưởng. Bên cạnh đó, hệ thống cũng phụ thuộc nhiều vào chất lượng kết nối mạng giữa client và server.

Hệ thống sử dụng mô hình kênh sự kiện kết hợp với mô hình dữ liệu tập trung để hình thành nên các không gian dữ liệu chia sẻ

- Client và server trao đổi qua Socket.IO bằng các sự kiện (join-document), submit-operation, document-operation, cursor-move, v.v.).
- Socket.IO đóng vai trò middleware điều phối, phát tán sự kiện tới các client đã subscribe phòng tài liệu.
- Client và server không truy xuất trực tiếp vào nhau mà tương tác qua event channel.

- Về mặt lưu trữ, nó cũng sử dụng **mô hình dữ liệu tập trung** ở backend:
 - Server là kho dữ liệu chung, client phải qua server để lấy/ghi trạng thái tài liệu.

IV. Giao thức trao đổi dữ liệu

1. Giao thức RESTful API

Giới thiệu

RESTful API là phương thức giao tiếp giữa Client và Server dựa trên giao thức HTTP theo mô hình Request–Response. Các tài nguyên trong hệ thống được quản lý thông qua các phương thức HTTP như GET, POST, PUT, PATCH và DELETE.

Cách thức áp dụng trong hệ thống

Trong hệ thống soạn thảo văn bản cộng tác, RESTful API được sử dụng cho các chức năng quản lý dữ liệu tài liệu và thông tin người dùng, bao gồm:

- Tạo tài liệu mới.
- Lấy danh sách tài liệu.
- Truy xuất nội dung tài liệu.
- Cập nhật thông tin tài liệu.
- Xóa tài liệu.
- Quản lý danh sách người dùng tham gia chỉnh sửa.

Các API được xây dựng bằng Express Framework trên nền Node.js và dữ liệu được lưu trữ trong MongoDB Atlas thông qua Mongoose.

Ví dụ:

GET /documents

POST /documents

PATCH /documents/:id

DELETE /documents/:id

Định dạng dữ liệu

Dữ liệu trao đổi giữa Client và Server sử dụng định dạng JSON (application/json) giúp dễ dàng xử lý và đồng bộ với Frontend React.

Ví dụ:

```
{"title": "Document A",  
  "content": "Hello world"}
```

Lý do lựa chọn

RESTful API phù hợp với các thao tác CRUD và quản lý tài liệu do:

- Dễ triển khai và bảo trì.
- Hoạt động ổn định.
- Dễ mở rộng hệ thống.
- Phù hợp với các thao tác không yêu cầu realtime.

Ưu điểm và nhược điểm

Ưu điểm:

- Đơn giản, dễ kiểm thử.
- Dễ tích hợp giữa Frontend và Backend.
- Quản lý tài nguyên rõ ràng.
- Phù hợp cho thao tác CRUD.

Nhược điểm:

- Không hỗ trợ cập nhật thời gian thực.
- Nếu dùng cho đồng bộ realtime sẽ phát sinh nhiều request gây quá tải hệ thống.

2. Giao thức WebSocket

Giới thiệu

WebSocket là giao thức truyền thông song công (Full-duplex) cho phép Client và Server duy trì kết nối liên tục để trao đổi dữ liệu theo thời gian thực.

Trong hệ thống, WebSocket được triển khai thông qua thư viện Socket.IO.

Cách thức áp dụng trong hệ thống

WebSocket được sử dụng cho chức năng chỉnh sửa cộng tác thời gian thực:

- Khi người dùng mở tài liệu, Client tham gia vào Room tương ứng của tài liệu.
- Các thao tác chỉnh sửa được gửi tới Server thông qua Socket.IO.
- Server xử lý và broadcast thay đổi tới các Client khác trong cùng Room.

Các sự kiện chính:

- join-document
- document-state
- active-users
- submit-operation
- operation-ack
- document-operation
- send-chat-message
- chat-message
- cursor-move
- remote-cursor
- leave-document
- disconnect

Định dạng dữ liệu

Dữ liệu được truyền dưới dạng Object chứa thông tin thao tác chỉnh sửa:

```
{"type": "insert",  
  "position": 5,
```

```
"text": "hello",  
"revision": 12}
```

Lý do lựa chọn

WebSocket phù hợp với hệ thống cộng tác thời gian thực vì:

- Độ trễ thấp.
- Hỗ trợ truyền dữ liệu hai chiều liên tục.
- Đồng bộ dữ liệu nhanh giữa nhiều người dùng.
- Giảm chi phí tạo lại kết nối liên tục như HTTP.

Ưu điểm và nhược điểm

Ưu điểm:

- Đồng bộ realtime nhanh.
- Hỗ trợ broadcast tới nhiều client.
- Tiết kiệm băng thông hơn polling.
- Phù hợp với collaborative editing.

Nhược điểm:

- Server phải duy trì kết nối liên tục.
- Quản lý socket phức tạp hơn HTTP.
- Cần xử lý reconnect khi mất mạng.

V. Kiểm thử và đánh giá

Các trường hợp thử nghiệm chính

1. Kết nối và join tài liệu

- Mở cùng một tài liệu trong 2-3 tab
- Kiểm tra sự kiện [join-document](#)
- Kỳ vọng:

- server trả document-state cho mỗi client
- từng client nhận được [revision](#), [content](#), [role](#)
- active users update đúng cho cả phòng

2. Đồng bộ thay đổi realtime cơ bản

- Gõ nội dung ở tab A
- Quan sát tab B có thay đổi ngay
- Kỳ vọng:
 - server nhận submit-operation
 - server emit document-operation tới các tab còn lại
 - tab B cập nhật nội dung mà không cần refresh
 - [revision](#) tăng cho mỗi thao tác

3. Test OT khi nhiều client cùng edit

- Tab A và tab B cùng sửa gần vị trí nhau:
 - A chèn “hello” ở vị trí 0
 - B chèn “world” ở vị trí 0
- Quan sát:
 - cả hai client cuối cùng đều có cùng nội dung hợp lệ
 - không bị double insert hoặc lệch vị trí
- Tương tự thử xóa/cắt dán cùng lúc:
 - A xóa trong một vùng
 - B chèn vào cùng vùng đó
- Kỳ vọng:
 - server dùng [transformSequence](#) và [applyOp](#)
 - client nhận document-operation và apply đúng

4. Xác thực operation-ack và revision

- Kiểm tra khi client gửi submit-operation:
 - nhận operation-ack chứa [appliedRevision](#)
 - nếu client có operation in-flight, nó phải đợi ack trước khi gửi tiếp
- Kỳ vọng:
 - [revision](#) cục bộ đồng bộ với server
 - client giữ lastAckContent đúng

5. Đồng bộ con trỏ và hiển thị người dùng

- Trong tab A di chuyển cursor / chọn vùng
- Tại tab B kiểm tra có nhận remote-cursor
- Kiểm tra active-users cập nhật khi user join/leave
- Kỳ vọng:
 - server broadcast remote-cursor
 - tab B hiển thị con trỏ màu / selection tương ứng
 - khi tab A đóng, cursor-clear và active-users cập nhật

6. Disconnect / reconnect / resync

- Ngắt kết nối tab A bằng cách đóng tab hoặc tắt mạng
- Mở lại, join document lại
- Trong thời gian mất kết nối, sửa nội dung ở tab B
- Kỳ vọng:
 - tab A reconnect và gửi request-resync nếu cần
 - server trả document-state
 - client merge hoặc resync đúng nội dung mới
 - không mất nội dung đã edit local nếu đã có cơ chế offline

7. Thử invalid operation

- Gửi thao tác không hợp lệ hoặc không đủ quyền từ client:
 - op có type sai
 - op thiếu field
 - role là viewer nhưng gửi edit
- Kỳ vọng:
 - server trả [operation-error](#)
 - client hiển thị lỗi và không áp dụng thay đổi

8. Quyền người dùng (owner/editor/viewer)

- Tạo ít nhất 2 người test với role khác nhau
- Kiểm tra:
 - viewer không được phép gửi submit-operation
 - editor/owner được phép chỉnh sửa

- role thay đổi được phản ánh [role-update](#)
- Kỳ vọng:
 - server kiểm tra role trong submit-operation
 - đổi role nếu dữ liệu thay đổi và notify client

Đánh giá kết quả

1. Tiêu chí Đạt

- Nội dung document đồng bộ đúng giữa client (với tốc độ dưới 40 từ/phút)
- Không có lỗi xung đột hiển nhiên khi edit đồng thời
- Revision tăng tuần tự và không bị trùng
- reconnect/resync hoạt động
- active users và cursor sync hiển thị đúng
- lỗi invalid operation được chặn
-

2. Tiêu chí Không đạt

- Document khác nhau giữa client sau khi edit
- Nếu xoá nhanh hoặc gõ nhanh (tốc độ tương đương khoảng trên 40 từ /phút) thì có hiện tượng xoá/gõ sót chữ
- Gõ tiếng việt thì thoảng bị duplicate